

保険約款ドラフト作成アプリケーション技術 選定レポート

エグゼクティブサマリー

本レポートは、保険約款ドラフト作成アプリケーションの技術選定のため、GitHub等で類似ツールを調査し、最適な技術スタックと実装方針を提案するものです。

主要な調査結果

- ・ **AI統合エディター**: Clineアーキテクチャが最も実用的で拡張性が高い
- ・ **ドキュメント生成**: Synapsy Writeのアプローチが保険約款作成に最適
- ・ **文書管理・検索**: PostgreSQL + pgvector + RAGの組み合わせが最もバランスが良い
- ・ **参考事例**: justInCaseの約款GitHub公開事例が透明性向上の参考になる

推奨技術スタック

- ・ **フロントエンド**: VSCode拡張機能（Clineベース）
- ・ **バックエンド**: Node.js/TypeScript + Express
- ・ **AI統合**: 複数AIプロバイダー対応
- ・ **データベース**: PostgreSQL + pgvector
- ・ **検索・RAG**: カスタムRAGエンジン

1. 調査背景と目的

1.1 プロジェクト要件

保険約款ドラフト作成アプリケーションに求められる要件は以下の通りです：

- 商品骨子からのドラフト作成**: 保険商品の基本情報から約款ドラフトを自動生成
- AI対話型インターフェース**: VSCode + Cline拡張機能のような生成AIとの対話環境
- 手動編集とAI支援の両立**: エディターでの直接編集とAIへの追加リクエストの両方に対応
- 参考約款の活用**: 過去の商品約款を参考にした類似保障の記載踏襲と言い回し統一

1.2 調査方法

GitHub、公式サイト、技術文書等を通じて以下の分野の技術調査を実施： - 類似ツール・技術の調査 - AI統合エディター技術の調査 - ドキュメント管理・検索技術の調査

2. 調査結果詳細

2.1 保険約款関連の先行事例

justInCase社の事例

2018年に日本初の保険約款GitHub公開を実施した株式会社justInCaseの事例は、保険業界におけるオープンソース的アプローチの先駆けとして注目されます。

特徴: - 保険約款をGitHubで公開し、プルリクエストを受付 - バージョン管理による約款の透明性向上 - エンジニアコミュニティからのフィードバック収集

学び: - 約款のバージョン管理の重要性 - オープンソース的アプローチの有効性 - コミュニティからの改善提案の価値

この事例から、約款作成プロセスの透明性と継続的改善の重要性が確認できます。

2.2 AI支援ドキュメント生成ツール

Synapsy Write

Next.jsベースのAI支援ドキュメント生成ツールで、保険約款作成アプリケーションの参考となる多くの機能を提供しています。

技術スタック: Next.js, OpenAI, Mistral, Anthropic **主要機能:** - 複数のAIプロバイダー対応による柔軟性 - テンプレート機能による複雑文書生成 - 会話モード（ChatGPT風）による対話的作成 - カスタマイズ可能なシステムプロンプト

保険約款作成への応用: - テンプレート機能を保険商品骨子に対応 - 複数AIプロバイダーによるコスト最適化 - 対話型インターフェースによる段階的作成支援

Mintlify Writer

VSCoDe拡張機能として提供されるAI支援ドキュメント作成ツールですが、現在は更新が停止されています。

特徴: - コードハイライトからドキュメント生成 - 複数プログラミング言語対応 - 複数ドキュメント形式対応（JSDoc, reST, NumPy等）

学び: - エディター統合の重要性 - コンテキスト理解による自動生成の可能性 - 継続的なメンテナンスの重要性

2.3 AI統合エディター技術

Cline

VSCode用AI自律コーディングエージェントとして160万インストールを達成し、AI統合エディターの成功事例として注目されます。

主要機能: - Plan/Actモード搭載による段階的作業 - ターミナル実行機能 - Model Context Protocol (MCP)対応 - Claude 4サポート（実験的） - 完全拡張可能なアーキテクチャ

保険約款作成への応用可能性: - Plan/Actモードを約款構造設計と内容作成に活用 - 拡張可能アーキテクチャによる保険業界特化機能追加 - 大規模ユーザーベースによる実証済み安定性

Continue.dev

AI-native開発環境として、カスタムAIコードアシスタントの作成・共有・利用を可能にします。

特徴: - VSCode/JetBrains対応 - カスタムAIコードアシスタント作成 - ルール、ツール、モデルのハブ - 複数LLM対応

保険約款作成への応用: - カスタムアシスタントによる保険業界特化機能 - 複数IDE対応による開発者の選択肢拡大 - エコシステム構築による機能拡張

2.4 ドキュメント管理・検索技術

Legal Document Assistant

RAG（Retrieval-Augmented Generation）を活用した法務文書アシスタントとして、保険約款作成における参考文書活用の参考となります。

技術スタック: - Python, Streamlit（UI） - PostgreSQL（構造化データ保存） - Elasticsearch（高速検索） - Google BERT LLM（Hugging Face API） - Airflow（ワークフロー管理） - Grafana（リアルタイム監視）

アーキテクチャの特徴: - PostgreSQLとElasticsearchの組み合わせによる効率的検索 - RAGアプローチによる高精度な文書検索 - リアルタイム監視による運用品質確保

保険約款作成への応用: - 過去約款データベースからの類似条項検索 - RAGによる文脈理解に基づく参考文書提案 - 段階的な約款作成プロセスの支援

ベクトルデータベース技術比較

Pinecone（マネージド）：- 高速類似性検索とスケーラビリティ - API簡易性による導入容易性 - 運用負荷の低さ - コスト面での課題

Chroma DB（オープンソース）：- 無料利用とカスタマイズ性 - RAG対応と軽量性 - 運用負荷とスケール課題

PostgreSQL + pgvector：- 既存DBインフラの活用 - SQL操作による柔軟性 - 運用知見の豊富さ - 専用DBと比較した性能課題

3. 技術選定と推奨アーキテクチャ

3.1 推奨技術スタック

フロントエンド・エディター統合

推奨: VSCode拡張機能（Clineアーキテクチャベース）

選定理由: 1. **実証済みの安定性**: 160万インストールによる大規模ユーザーベース 2. **拡張性**: オープンソースアーキテクチャによる保険業界特化機能追加 3. **ユーザビリティ**: エンジニア・非エンジニア両方に親しみやすいUI 4. **エコシステム**: VSCodeの豊富な拡張機能との連携

AI・LLM統合

推奨: 複数AIプロバイダー対応（Synapsy Writeアプローチ）

選定理由: 1. **リスク分散**: 単一プロバイダー依存の回避 2. **コスト最適化**: 用途別最適モデル選択 3. **機能最適化**: OpenAI（汎用）、Anthropic（長文）、Mistral（コスト効率）等の使い分け 4. **将来性**: 新しいモデルへの対応容易性

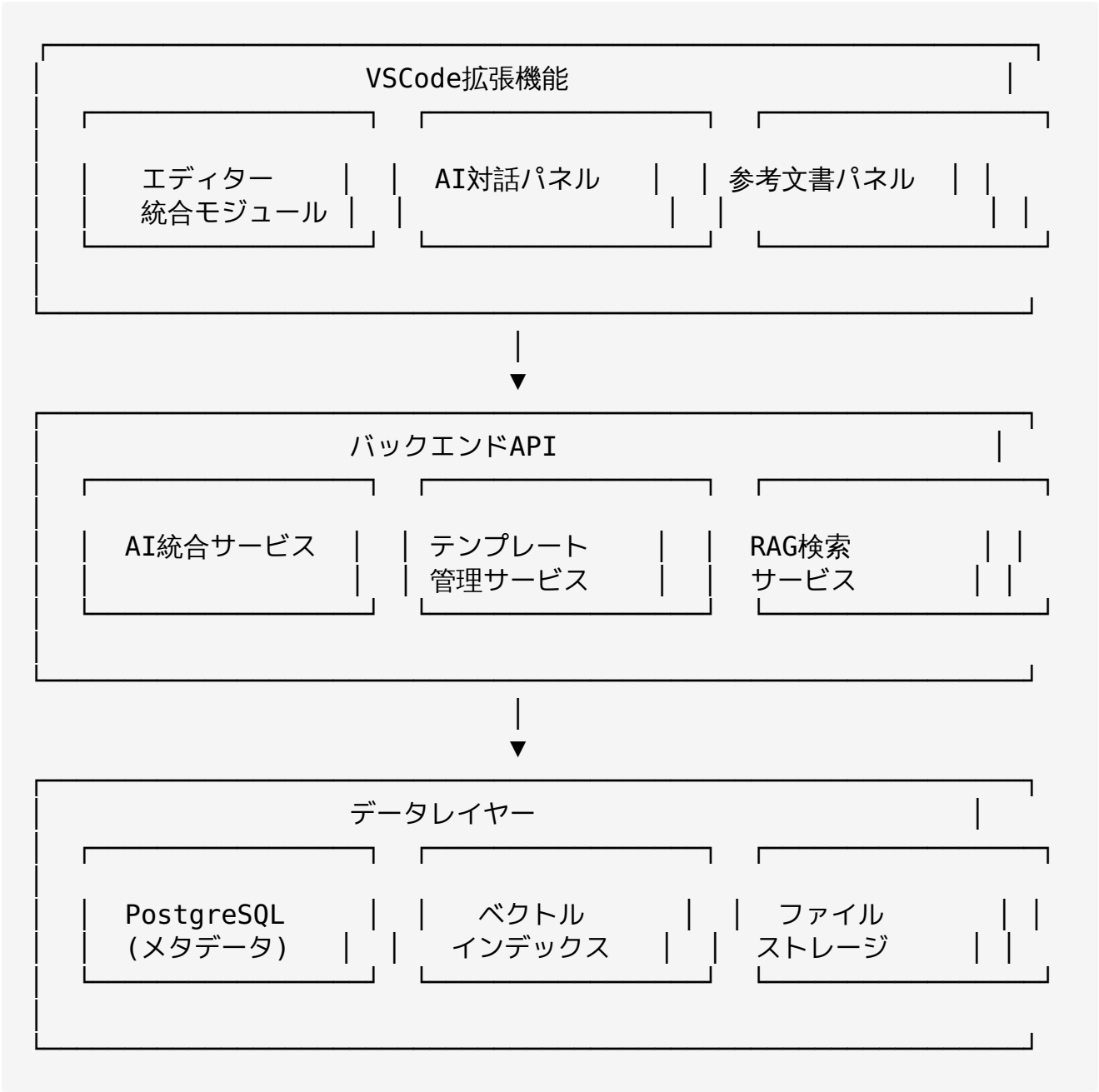
ドキュメント管理・検索

推奨: PostgreSQL + pgvector + カスタムRAGエンジン

選定理由: 1. **統合容易性**: 既存システムとの親和性 2. **運用効率**: SQL操作による柔軟なクエリ 3. **コスト効率**: オープンソースによる運用コスト最適化 4. **拡張性**: pgvectorによるベクトル検索機能

3.2 システムアーキテクチャ設計

全体アーキテクチャ



モジュール設計

- 1. エディター統合モジュール** - VSCode拡張機能のコア機能 - リアルタイム編集支援 - シンタックスハイライト（約款特化） - 自動保存・バージョン管理
- 2. AI統合モジュール** - 複数AIプロバイダー管理 - プロンプト最適化 - レスポンス品質管理 - コスト監視
- 3. テンプレート管理モジュール** - 保険商品骨子テンプレート - 約款構造テンプレート - カスタムテンプレート作成 - テンプレート継承機能

4. RAG検索モジュール - 参考約款検索 - 類似条項抽出 - 文脈理解による推奨 - 検索結果ランキング

5. ドキュメント管理モジュール - 約款バージョン管理 - 変更履歴追跡 - 承認ワークフロー - エクスポート機能

3.3 実装フェーズ計画

Phase 1: 基盤構築（3-4ヶ月）

- ・ VSCode拡張機能の基本フレームワーク
- ・ バックエンドAPI基盤
- ・ PostgreSQL + pgvectorセットアップ
- ・ 基本的なAI統合機能

Phase 2: コア機能開発（4-5ヶ月）

- ・ テンプレートエンジン実装
- ・ RAG検索機能実装
- ・ AI対話インターフェース
- ・ 基本的な約款生成機能

Phase 3: 高度機能・最適化（3-4ヶ月）

- ・ 参考約款データベース構築
- ・ 高度なRAG機能
- ・ UI/UX最適化
- ・ パフォーマンス最適化

Phase 4: 運用・保守体制（継続）

- ・ 監視・ログ機能
- ・ セキュリティ強化
- ・ ユーザーフィードバック対応
- ・ 継続的改善

4. 技術的考慮事項

4.1 セキュリティ・コンプライアンス

- ・ **データ暗号化:** 保存時・転送時の暗号化
- ・ **アクセス制御:** ロールベースアクセス制御
- ・ **監査ログ:** 全操作の記録・追跡

- ・ **プライバシー保護:** 個人情報の適切な取り扱い

4.2 スケーラビリティ

- ・ **水平スケーリング:** マイクロサービスアーキテクチャ
- ・ **キャッシュ戦略:** Redis活用による高速化
- ・ **CDN活用:** 静的リソースの配信最適化
- ・ **負荷分散:** ロードバランサーによる可用性確保

4.3 運用・保守性

- ・ **監視体制:** Grafana/Prometheusによる監視
- ・ **ログ管理:** 構造化ログによる問題特定
- ・ **自動化:** CI/CDパイプラインによる品質確保
- ・ **ドキュメント:** 技術文書の継続的更新

5. リスク分析と対策

5.1 技術的リスク

AI依存リスク

リスク: AIサービスの障害や品質低下 **対策:** - 複数AIプロバイダーによるフェイルオーバー - ローカルLLMのバックアップ運用 - 人手による品質チェック体制

データ品質リスク

リスク: 参考約款データの品質や網羅性不足 **対策:** - 段階的なデータ品質向上 - 専門家によるデータ監修 - ユーザーフィードバックによる継続改善

技術的負債リスク

リスク: 急速な技術進歩による陳腐化 **対策:** - モジュラー設計による部分更新 - オープンソース技術の積極活用 - 定期的な技術スタック見直し

5.2 運用リスク

セキュリティリスク

リスク: 機密情報の漏洩や不正アクセス **対策:** - 多層防御によるセキュリティ強化 - 定期的なセキュリティ監査 - インシデント対応体制の整備

法的・コンプライアンスリスク

リスク: 生成された約款の法的妥当性 **対策:** - 法務専門家による最終チェック必須化 - 免責事項の明確化 - 継続的な法改正対応

5.3 ビジネスリスク

ユーザー受容性リスク

リスク: 従来の作業方法からの移行抵抗 **対策:** - 段階的な導入とトレーニング - 既存ワークフローとの統合 - ユーザビリティの継続改善

6. 成功指標とKPI

6.1 開発効率指標

- ・ **約款作成時間:** 従来比50%以上の短縮
- ・ **品質向上:** レビュー指摘事項30%以上削減
- ・ **一貫性向上:** 用語・表現の統一率90%以上

6.2 システム性能指標

- ・ **応答時間:** AI生成レスポンス5秒以内
- ・ **可用性:** 99.9%以上のアップタイム
- ・ **検索精度:** 関連文書検索の適合率80%以上

6.3 ユーザー満足度指標

- ・ **ユーザー満足度:** NPS（Net Promoter Score）70以上
- ・ **採用率:** 対象ユーザーの80%以上が継続利用
- ・ **学習効果:** 新規ユーザーの習熟期間2週間以内

7. 結論と次のステップ

7.1 技術選定結論

本調査により、保険約款ドラフト作成アプリケーションに最適な技術スタックとして以下を推奨します:

コア技術スタック: - **フロントエンド:** VSCode拡張機能（Clineアーキテクチャベース） - **バックエンド:** Node.js/TypeScript + Express - **AI統合:** 複数AIプロバイダー対応（OpenAI,

Anthropic, Mistral) - データベース: PostgreSQL + pgvector - 検索・RAG: カスタムRAGエンジン

この選定により、要件である「VSCode + Cline拡張機能のような生成AIとの対話環境」を実現しつつ、保険業界特有のニーズに対応できる拡張性と実用性を確保できます。

7.2 重要な成功要因

1. 段階的な実装: 基盤から高度機能まで段階的な開発
2. ユーザー中心設計: 実際の約款作成業務フローとの整合
3. 品質保証体制: AI生成内容の人的チェック体制
4. 継続的改善: ユーザーフィードバックによる機能改善

7.3 次のステップ

1. 詳細要件定義: 具体的な機能要件と非機能要件の策定
2. プロトタイプ開発: 核となる機能のPoC（Proof of Concept）実装
3. ユーザーテスト: 実際の約款作成担当者による評価
4. 本格開発計画: 詳細な開発スケジュールとリソース計画

本技術選定により、効率的で高品質な保険約款ドラフト作成を支援し、保険業界のデジタル変革に貢献できるアプリケーションの実現が期待されます。

レポート作成日: 2025年6月6日

調査期間: 2025年6月6日

調査範囲: GitHub、公式サイト、技術文書等による公開情報調査